



山东大学

信息科学与工程学院

2025—2026 学年第一学期

实 验 报 告

课程名称: Java 编程技术

实验名称: 第四次实验

专 业 班 级 通信三班

学 生 学 号

学 生 姓 名

实 验 时 间 2025 年 10 月 25 日

实 验 报 告

【实验目的】

- 1.熟悉 Java 语言的数组与字符串
- 2.熟悉 Java 语言的类与对象

【实验要求】

一、在 Eclipse 平台中实现下列简单的程序

【实验具体内容】

1、抽象类的练习

编写一个程序，在程序中定义一个抽象类 Shape，再定义两个抽象方法 Area 和 printArea，定义两个 Shape 类的子类 Rectangle 和 Circle 类，在子类中实现父类的抽象方法。

(1) 实验源程序

```
package cn;

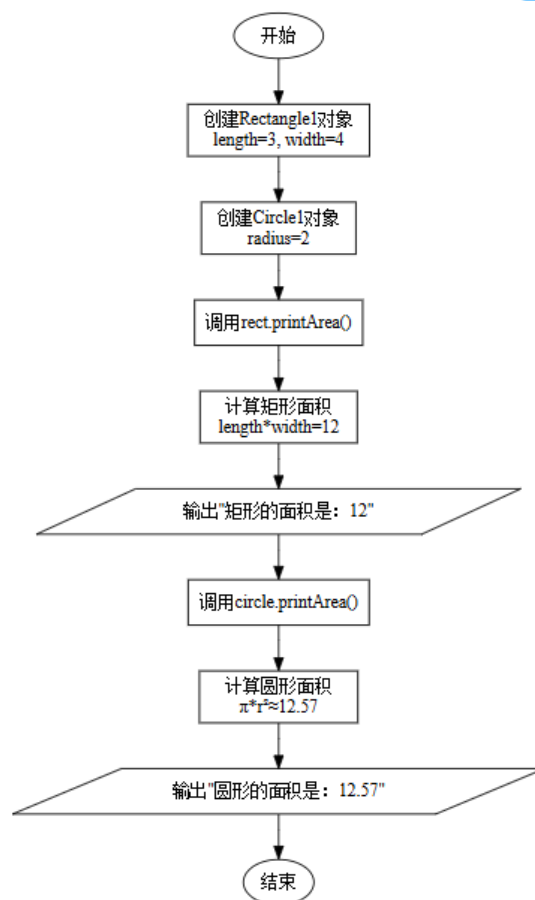
public abstract class Shape {
    public abstract double Area();
    public abstract void printArea();
    public static void main(String[] args) {
        Shape rect = new Rectangle1(3, 4);
        Shape circle = new Circle1(2);
        rect.printArea();
        circle.printArea();
    }
}

class Rectangle1 extends Shape {
    private double length;
    private double width;
    public Rectangle1(double length, double width) {
        this.length = length;
        this.width = width;
    }
    @Override
    public double Area() {
        return length * width;
    }
    @Override
```

```

    public void printArea() {
        System.out.println("矩形的面积是: " + Area());
    }
}
class Circle1 extends Shape {
    private double r;
    public Circle1(double r) {
        this.r = r;
    }
    @Override
    public double Area() {
        return Math.PI * r * r; // 修正为正确的圆形面积公式
    }
    @Override
    public void printArea() {
        System.out.println("圆形的面积是: " + Area());
    }
}
    
```

(2) 代码流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验代码思路：这是同实验 A 一样的功能，但是在再次实验，我们需要实现一个抽象类，其中有两个抽象方法，再使一个类继承抽象类，再其中分别重写两个抽象方法，以实现预期的功能。

1) 编写 java 代码并运行

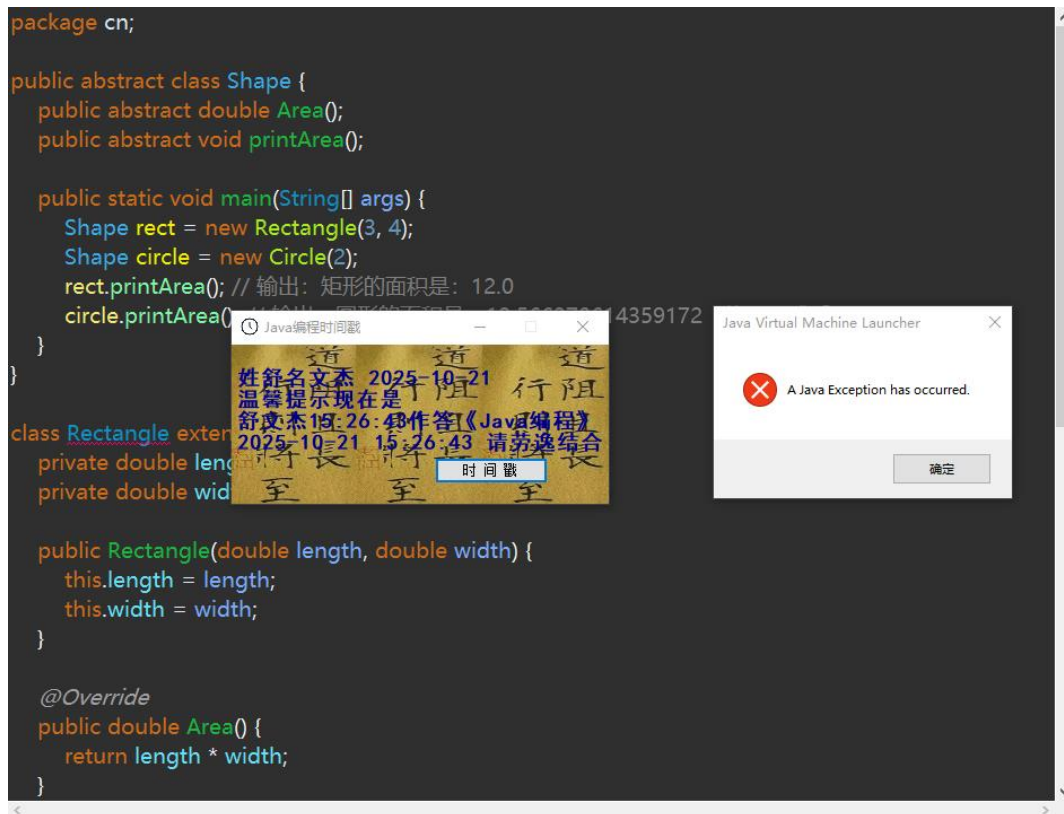


图 1 报错：该类已存在

？，我明明只在这一个地方定义了 Rectangle 类，它为什么会报错这个类已存在呢？这个疑问很快被我解决了，我这个文件是在 package cn 下的，这个类与我前面的实验所用的类重名了，应该换一个名字就成功了。

2) 更换名字后，我再次进行实验

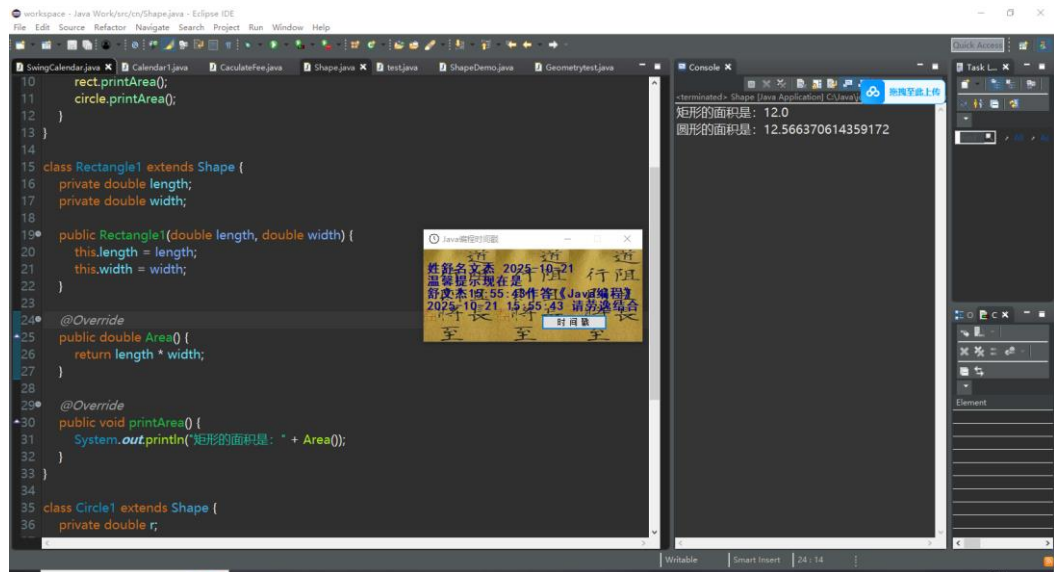


图 2 实验成功截图

可以发现，我们的实验结果是符合预期的，实验成功。但是这又引起了我的疑问，为什么在 Java 怎么一个连变量都要进行隐藏以保证扩展性的语言中，为什么这种问题会出现呢？

查阅资料后，我发现“同一包下类已存在”的错误，本质是 Java 通过“全限定名”唯一标识类的机制导致的——同一包下的同名类会使全限定名重复，违反了类的唯一标识规则，因此编译器会直接报错。

2、编写下面程序写出运行结果，谈谈成员变量的继承与隐藏，方法的覆盖与重载。

(1) 实验源程序

```
package cn;

public class test {
    public static void main(String[] arg) {
        B m = new B();
        m.sum1();
        m.sum2();
        m.sum2(50);
        m.sum3(50);
    }
}

class A {
    int sum, num1, num2;
    public A() {
        num1 = 10;
    }
}
```

```

        num2 = 20;
        sum = 0;
    }
    void sum1() {
        sum = num1 + num2;
        System.out.println("sum=" + num1 + "+" + num2 + "=" + sum);
    }
    void sum2(int n) {
        num1 = n;
        sum = num1 + num2;
        System.out.println("sum=" + num1 + "+" + num2 + "=" + sum);
    }
}
class B extends A {
    int num2;
    public B() {
        num2 = 200;
    }
    void sum2() {
        sum = num1 + num2;
        System.out.println("sum=" + num1 + "+" + num2 + "=" + sum);
    }
    void sum2(int n) {
        num1 = n;
        sum = num1 + num2;
        System.out.println("sum=" + num1 + "+" + num2 + "=" + sum);
    }
    void sum3(int n) {
        super.sum2(n);
        System.out.println("sum=" + num1 + "+" + num2 + "=" + sum);
    }
}

```

(2) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验代码思路： 这里与上一次实验一致，我们需要考虑变量的隐藏与重载，对于第一个 sum，虽然 B 继承了 A，但是它并没有与其相对应的 sum1 方法，因此不会进行重写，输出应该就是 A 类的 num1+num2 即为 30，对于 sum2，它同时是 A 中方法的重写，也存在两种重载方式，对于第一种，我们直接将 A 中初始化的 num1+B 中重写的 num2 结果即为 210，对于第二种，我们对 num1 进行重写，其值是传入的参数，因此结果为 250。最后的 sum3，它使用了 super，因此调用的是 A 类中的方法，最后的结果 sum 应该是 70，但是这里会同时输出两行结果一个是 20 + 50 =70，另

一个是 $200 + 50 = 70$ 。

1) 编写 java 代码并运行

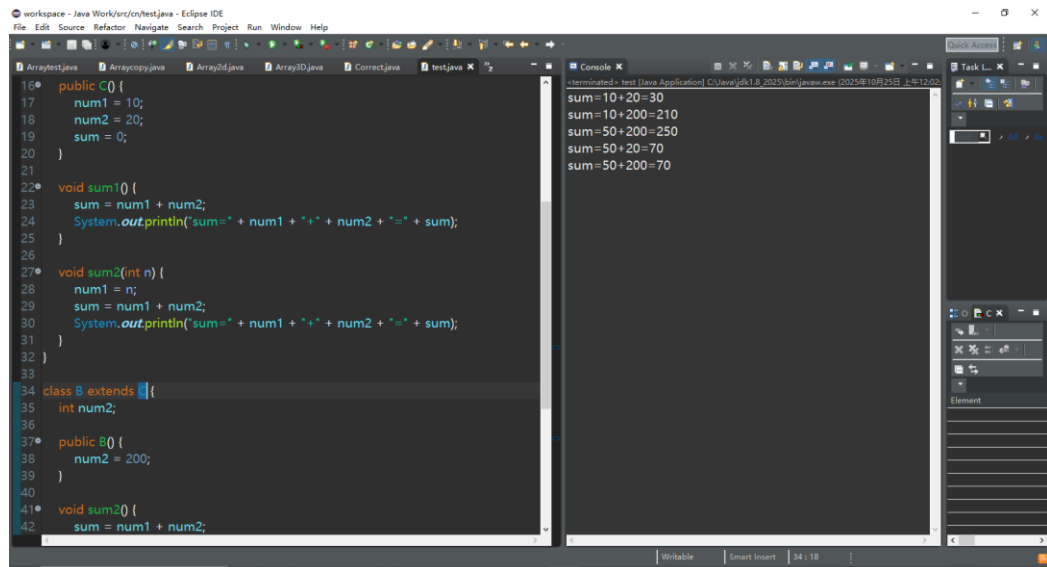


图 3 实验结果截图

可以发现，运行结果符合我们的在思路中所述，但这里其实非常绕，我们可以简洁地叙述，当一个类继承了另一个类，也就继承它父类的所有属性与方法，除非我们对某几个属性与方法进行重写，它的其他值仍为其父类的属性初始化的值。

3、理解下面 3 个类的定义，分析它们之间的关系，写出运行结果。

(1) 实验源程序

```

1: class SuperClass {
2:     int x;
3:     SuperClass() {
4:         x=3;
5:         System.out.println("in SuperClass : x=" +x);
6:     }
7:     void doSomething() {
8:         System.out.println("in SuperClass.doSomething()");
9:     }
10: }
    
```

```

1:  class SubClass extends SuperClass {
2:      int x;
3:      SubClass() {
4:          super(); //调用父类的构造方法
5:          x=5; //super() 要放在方法中的第一句
6:          System.out.println("in SubClass :x="+x);
7:      }
8:      void doSomething( ) {
9:          super.doSomething(); //调用父类的方法
10:         System.out.println("in SubClass.doSomething()");
11:         System.out.println("super.x="+super.x+" sub.x="+x);
12:     }
13: }

1:  import java.io.*;
2:  public class Inheritance {
3:      public static void main(String[] args) {
4:          SubClass subC=new SubClass();
5:          subC.doSomething();
6:      }
7:  }

```

```

public class HelloJavaWorld {
    // 成员变量
    private int instanceVar;

    // 静态成员变量
    private static int staticVar;

    // 参数变量
    public void method(int paramVar,int instanceVar) {
        // 局部变量
        int localVar = 10;

        // 变量赋值
        this.instanceVar = instanceVar;
        staticVar = paramVar;

        System.out.println("成员变量:" + this.instanceVar);
        System.out.println("静态变量:" + staticVar);
        System.out.println("参数变量:" + paramVar);
        System.out.println("局部变量:" + localVar);
    }

    public static void main(String[] args) {
        new HelloJavaWorld().method(20,30);
    }
}
    
```

(2) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

首先，由于子类继承了父类，当子类被 new 出来后，会自动调用父类的构造方法，因此第一句话是 in SuperClass x= 3，于是第二句话则为 in SubClass x= 5
 由于在主类中调用 SubClass 的 dosomething 的方法，因此我们看 SubClass 的方法：

```

8: void doSomething() {
9:     super.doSomething(); //调用父类的方法
10: System.out.println("in SubClass.doSomething()");
11: System.out.println("super.x="+super.x+" sub.x="+x);
12: }
13: }
    
```

那么我再看看父类的方法：（注意，这里的 x = 3）

```

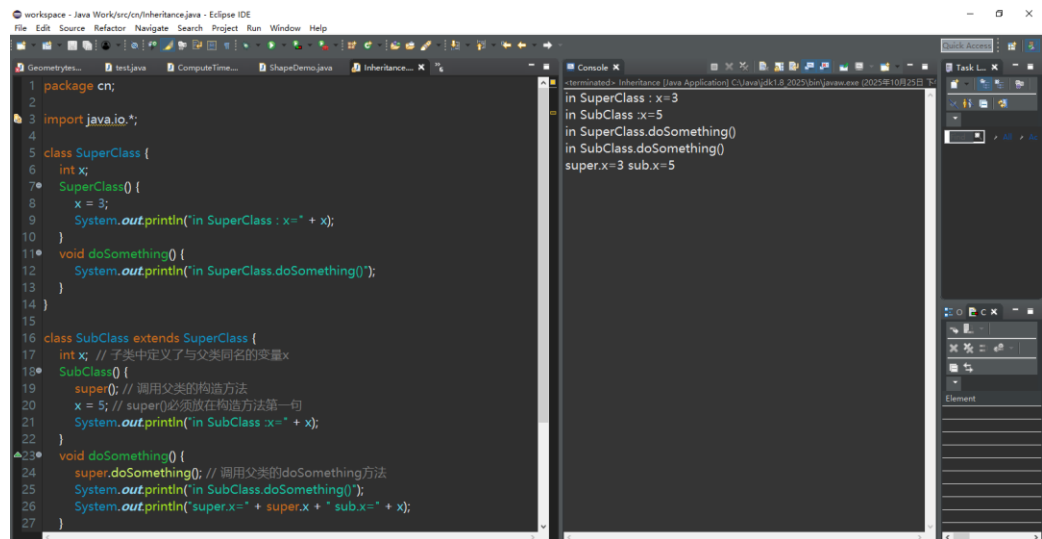
7: void doSomething() {
8:     System.out.println("in SuperClass.doSomething()");
9: }
    
```

，第三个输出的是： in SuperClass.doSomething()

第四行输出即：in SubClass.doSomething()

第五行输出：super.x = 3 , sub.x =5

我们执行一下来验证：



The screenshot shows the Eclipse IDE with a Java project. The code in the editor is as follows:

```

1 package cn;
2
3 import java.io.*;
4
5 class SuperClass {
6     int x;
7     SuperClass() {
8         x = 3;
9         System.out.println("in SuperClass : x=" + x);
10    }
11    void doSomething() {
12        System.out.println("in SuperClass.doSomething()");
13    }
14 }
15
16 class SubClass extends SuperClass {
17     int x; // 子类中定义了与父类同名的变量x
18     SubClass() {
19         super(); // 调用父类的构造方法
20         x = 5; // super()必须放在构造方法第一句
21         System.out.println("in SubClass :x=" + x);
22     }
23     void doSomething() {
24         super.doSomething(); // 调用父类的doSomething方法
25         System.out.println("in SubClass.doSomething()");
26         System.out.println("super.x=" + super.x + " sub.x=" + x);
27     }
28 }

```

The console output on the right shows the following sequence of lines:

```

in SuperClass : x=3
in SubClass :x=5
in SuperClass.doSomething()
in SubClass.doSomething()
super.x=3 sub.x=5

```

可以发现，我们的理解是正确的。

```

public class HelloJavaWorld {
    // 成员变量
    private int instanceVar;

    // 静态成员变量
    private static int staticVar;

    // 参数变量
    public void method(int paramVar,int instanceVar) {
        // 局部变量
        int localVar = 10;

        // 变量赋值
        this.instanceVar = instanceVar;
        staticVar = paramVar;

        System.out.println("成员变量:" + this.instanceVar);
        System.out.println("静态变量:" + staticVar);
        System.out.println("参数变量:" + paramVar);
        System.out.println("局部变量:" + localVar);
    }

    public static void main(String[] args) {
        new HelloJavaWorld().method(20,30);
    }
}

```

这里我们的我们定义了两个变量，一个是成员变量 instanceVar，另一个是静态变量 staticVar, 为什么被 static 修饰的属性不能使用 this 引出呢？这是因为被 static 修饰的属性属于全体类,而 this 只能作用于这个类实例化后的某一对象,因此使用 this 作用静态变量是错误的。（同样，这对 super 也是成立的）

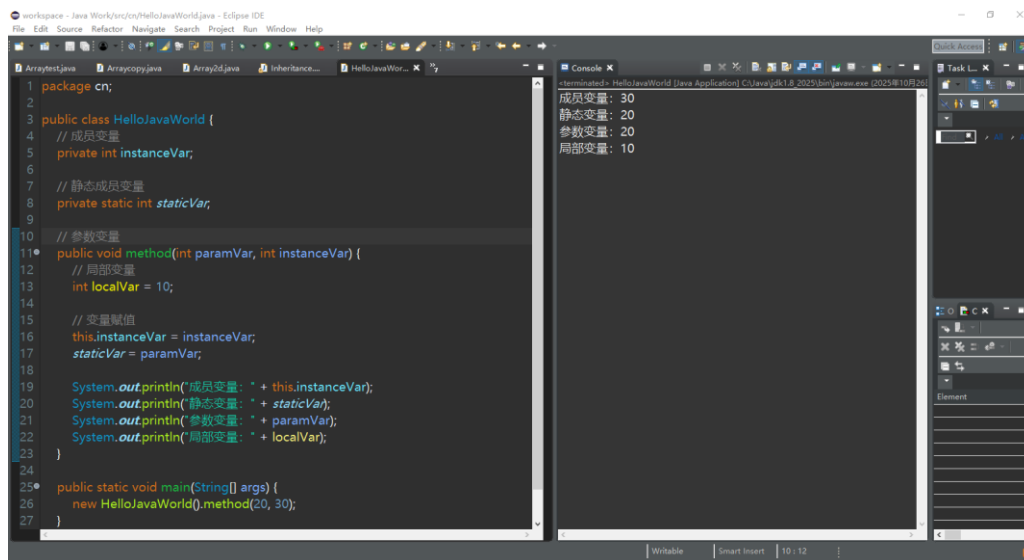


图 4 运行结果截图

通过运行结果我们可以发现，这里的静态变量的生命周期是最长的，它应该和类同存，而作为成员变量的 `instanceVar` 是随着对象同存，参数变量与局部变量则是随着方法同存。

4、接口和继承的综合应用

首先定义一个接口（`IShapeArea`），其中包含返回面积的方法（`getArea`）。然后定义一个矩形类（`Rectangle`）和一个圆类（`Circle`），并派生出一个正方形类（`Square`），即正方形的父类为矩形类，三者都要求实现接口 `IShapeArea`，自行扩充成员变量和方法（比如周长和面积）。

在主方法 `main` 中建一数组，数组中放入一些上述类型的对象，并计算它们的周长和面积之和。

（1）实验源程序

```
package cn;

interface IShapeArea{
    double getArea();
}

class Rectangle2 implements IShapeArea{
    private double length;
    private double width;
```

```

    public Rectangle2(double length, double width) {
        this.length = length;
        this.width = width;
    }

    public double getPerimeter() {
        return (length+width)*2;
    }

    @Override
    public double getArea() {
        return length*width;
    }
}

class Circle2 implements IShapeArea{
    private double r;

    public Circle2(double r) {
        this.r = r;
    }

    public double getPerimeter() {
        return Math.PI*r*2;
    }

    @Override
    public double getArea() {
        return Math.PI*r*r;
    }
}

class Square implements IShapeArea{
    private double a;

    public Square(double a) {
        this.a = a;
    }

    public double getPerimeter() {
        return 4*a;
    }

    @Override
    public double getArea() {
        return a*a;
    }
}

```

```

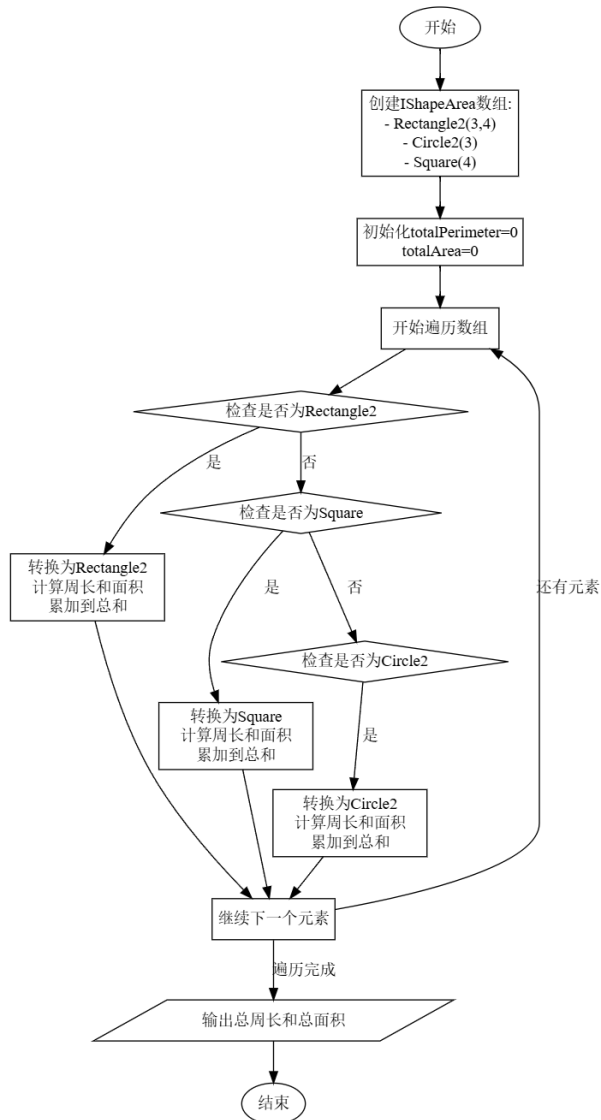
    }
}
public class ShapeDemo {
    public static void main(String[] args) {
        IShapeArea[] shapes = {
            new Rectangle2(3,4),
            new Circle2(3),
            new Square(4)
        };

        double totalPerimeter = 0;
        double totalArea = 0;

        for (IShapeArea shape : shapes) {
            if (shape instanceof Rectangle2) {
                Rectangle2 rect = (Rectangle2) shape;
                totalPerimeter += rect.getPerimeter();
                totalArea += rect.getArea();
            }
            else if (shape instanceof Square) {
                Square square = (Square) shape;
                totalPerimeter += square.getPerimeter();
                totalArea += square.getArea();
            }
            else if (shape instanceof Circle2) {
                Circle2 circle = (Circle2) shape;
                totalPerimeter += circle.getPerimeter();
                totalArea += circle.getArea();
            }
        }
        System.out.println("所有图形的周长之和: " + totalPerimeter);
        System.out.println("所有图形的面积之和: " + totalArea);
    }
}

```

(2) 代码流程图与知识点总结



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验思路：这个实验与第一个实验内容非常类似，因为抽象类其实是接口的一个过渡形式。接口与抽象类不同的是，他所定义的抽象方法不需要显示声明使用 `abstract` 修饰，同时接口实现的抽象方法默认是 `public` 的，而抽象类需要显式声明 `public` 才行。因此，我们只需要定义一个接口，再定义不同的类来实现这个接口，令它们各自重写一遍 `getArea` 方法即可。最后使用 `foreach` 语句遍历接口，使用 `instance` 语句将三个类分离开，分别调用各自的方法即可（这是因为在接口中我们没有定义 `getPerimeter` 方法，不能直接点出来）。

1) 编写 java 代码并运行

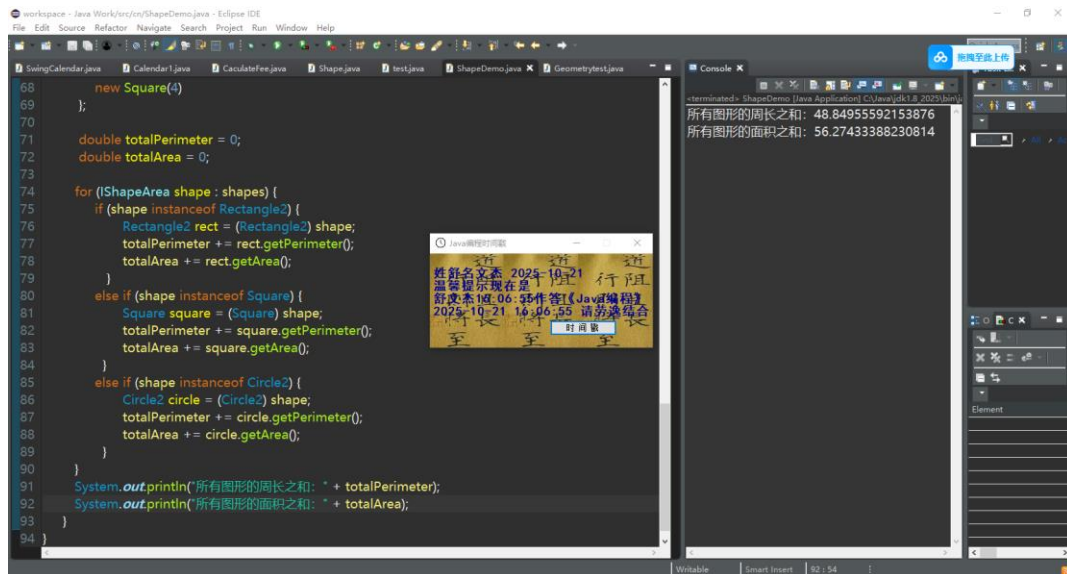


图 5 实验结果截图

可以发现，我们的运行结果符合预期，我们的实验成功。

5、接口的小小综合应用（必须采用 Java 的反射方法来实现）

为某研究所编写一个通用程序，用来计算每一种交通工具运行 1000 米所需的时间，已知每种交通工具的参数都是 3 个整数 A、B、C 的表达式。现有两种工具：Car 和 Plane，其中 Car 的速度运算公式为： $A*B/C$ ，Plane 的速度运算公式为： $A+B+C$ 。需要编写 4 类：ComputeTime、Plane、Car 和接口（ICommon），要求在未来如果增加第 3 种交通工具的时候，不必修改以前的任何程序，只需要编写新的交通工具的程序即可。

（计算结果如果有小数，仅保留小数点后两位）

（1）实验源程序

```
interface ICommon {
    double calculateSpeed(int a, int b, int c);
}

class Car implements ICommon {
    @Override
    public double calculateSpeed(int a, int b, int c) {
        if (c <= 0) {
            System.out.println("除数不能为 0");
            return 0;
        } else {
            return a * b / (double) c;
        }
    }
}
```

```

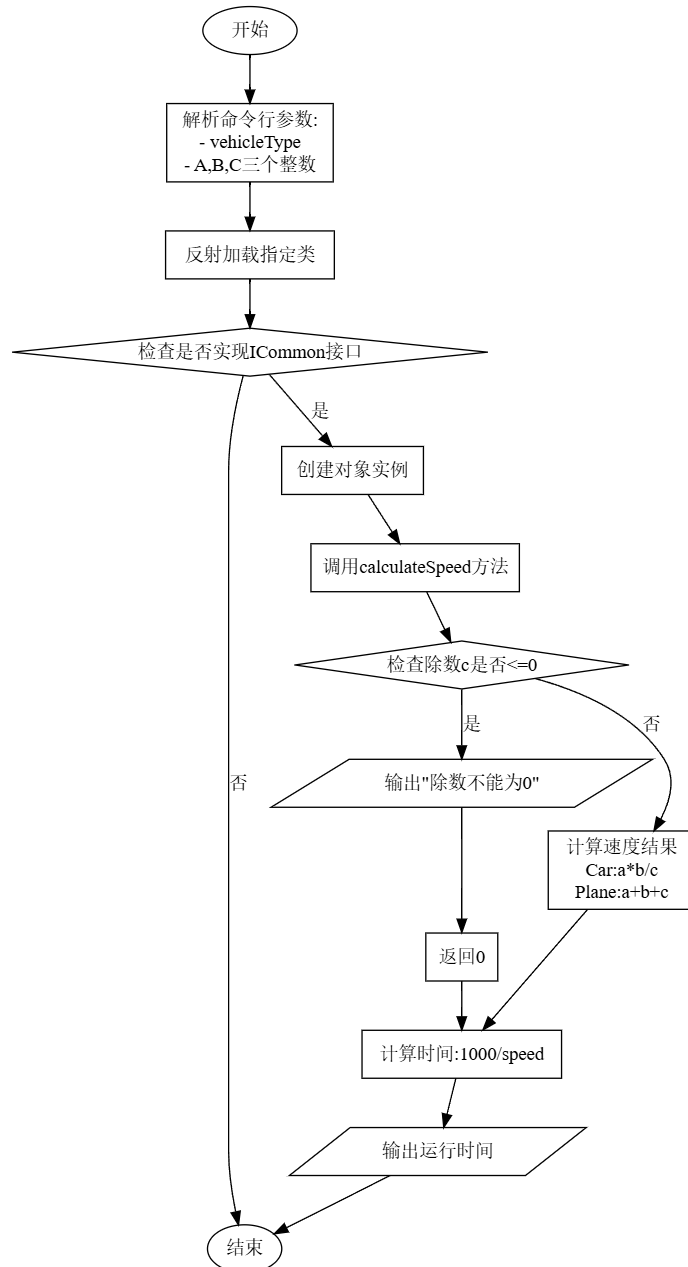
    }
    class Plane implements ICommon {
        @Override
        public double calculateSpeed(int a, int b, int c) {
            return a + b + (double) c;
        }
    }
}

public class ComputeTime {
    public static void main(String[] args) throws Exception {

        String vehicleType = args[0];
        int A = Integer.parseInt(args[1]);
        int B = Integer.parseInt(args[2]);
        int C = Integer.parseInt(args[3]);
        // 反射加载类
        Class<?> clazz = Class.forName(vehicleType);
        // 校验是否实现 ICommon 接口
        if (ICommon.class.isAssignableFrom(clazz)) {
            // 实例化对象
            ICommon vehicle = (ICommon) clazz.newInstance();
            // 计算速度和时间
            double speed = vehicle.calculateSpeed(A, B, C);
            double time = 1000.0 / speed;
            // 输出结果
            System.out.printf("运行 1000 米所需时间: %.2f 单位时间\n", time);
        }
    }
}

```

(2) 代码流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验代码思路：反射这个概念老师只是在课上随便提了一嘴，初看这个概念，我是一头雾水，向 AI 询问了反射的概念后，我初步了解到反射是程序在运行时不知道对应的类以及它的方法，只是在运行时加载，因此我们就不需要修改完接口的代码后每次修改主程序的代码，这种机制类似 python 中的变量属性的动态，不需要提前声明，仅在运算时辨别。

简单来说，在没有使用反射前，我们使用接口未定义的方法前，我们需要使用 `instance` 语句分开了调用方法，即：

```
if (shape instanceof Rectangle2) {
```

```

        Rectangle2 rect = (Rectangle2) shape;
        totalPerimeter += rect.getPerimeter();
        totalArea += rect.getArea();
    }
    else if (shape instanceof Square) {
        Square square = (Square) shape;
        totalPerimeter += square.getPerimeter();
        totalArea += square.getArea();
    }
    else if (shape instanceof Circle2) {
        Circle2 circle = (Circle2) shape;
        totalPerimeter += circle.getPerimeter();
        totalArea += circle.getArea();
    }
}

```

反射的核心语句就是定义一个不确定的类，再在确定其实现了 `ICommon` 类后对其进行实例化，最后调用该类的方法，核心语句只有两个：

```

Class<?> clazz = Class.forName(vehicleType);
ICommon vehicle = (ICommon) clazz.newInstance();

```

其他的部分就是与上一个实验一样了，就是实现接口重写 `calculateSpeed`。

1) 编写 java 代码并运行

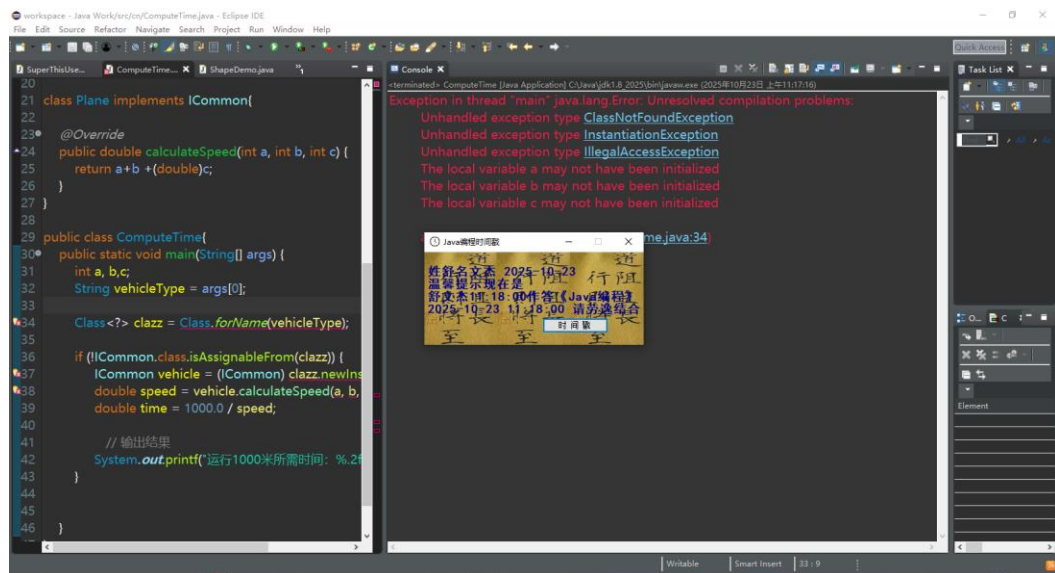


图 6 提示缺少异常处理

？，我的代码应该没有问题，为什么会编译不通过呢，向 AI 提问后，他告诉我所有的反射均应该异常处理，因为在调用反射 API 时就会抛出异常（Checked Exception），因此我们需要进行异常处理。这里根据课上所讲，我们直接在主类抛出一个省力的异常：

```
public static void main(String[] args) throws Exception {
```

这样可以抛出所有的异常了。

(2) 异常处理完成后，继续编译，在 Dos 环境下运行

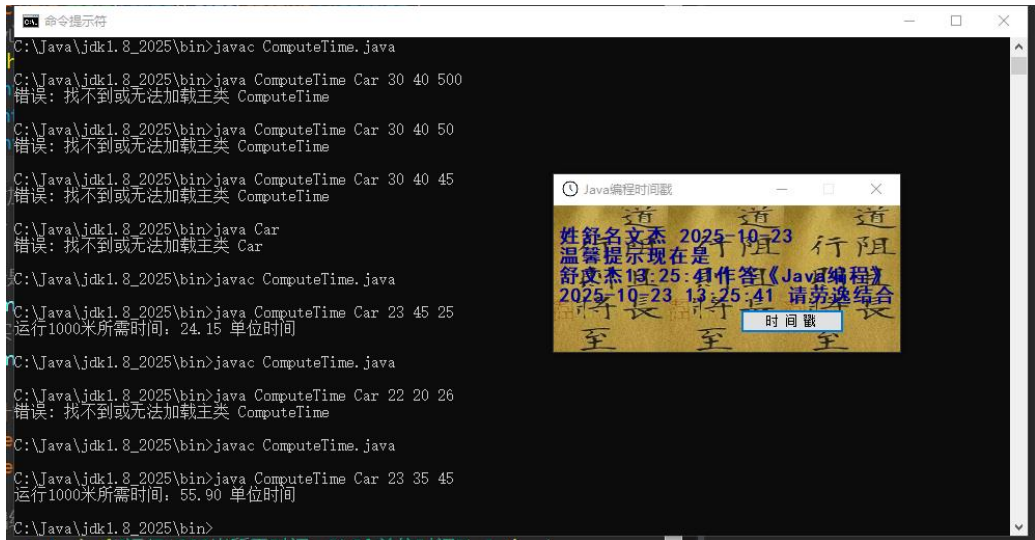


图 7 Dos 环境下运行结果截图

(3) 此时，我们通过 Notepad++ 编辑 Java 代码，新添加 Bike 类，试验我们的反射机制

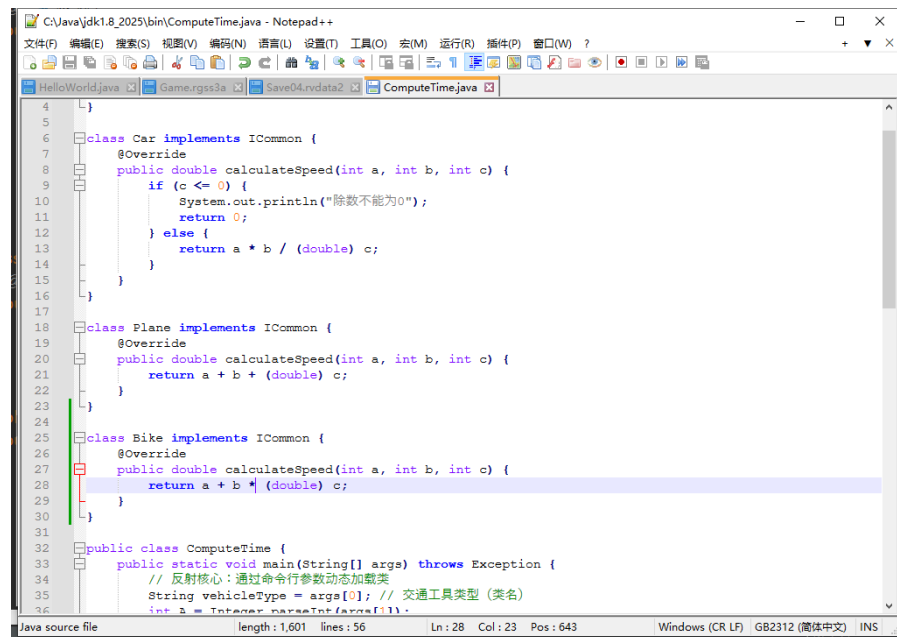


图 8 添加 Bike 类

```

C:\Java\jdk1.8_2025\bin>java ComputeTime Car 30 40 50
错误: 找不到或无法加载主类 ComputeTime
C:\Java\jdk1.8_2025\bin>java ComputeTime Car 30 40 45
错误: 找不到或无法加载主类 ComputeTime
C:\Java\jdk1.8_2025\bin>java Car
错误: 找不到或无法加载主类 Car
C:\Java\jdk1.8_2025\bin>javac ComputeTime.java
C:\Java\jdk1.8_2025\bin>java ComputeTime Car 23 45 25
运行1000米所需时间: 24.15 单位时间
C:\Java\jdk1.8_2025\bin>javac ComputeTime.java
C:\Java\jdk1.8_2025\bin>java ComputeTime Car 22 20 26
错误: 找不到或无法加载主类 ComputeTime
C:\Java\jdk1.8_2025\bin>javac ComputeTime.java
C:\Java\jdk1.8_2025\bin>java ComputeTime Car 23 35 45
运行1000米所需时间: 55.90 单位时间
C:\Java\jdk1.8_2025\bin>javac ComputeTime.java
C:\Java\jdk1.8_2025\bin>java ComputeTime Bike 10 12 20
运行1000米所需时间: 4.00 单位时间
C:\Java\jdk1.8_2025\bin>
    
```

可以发现，Bike 类被正确加载出来了，因此我们的反射机制构建成功。

【实验心得】

知识点总结:

1. 抽象类:

抽象类是用 `abstract` 修饰的类，包含抽象方法（无方法体）和非抽象方法（可直接继承）。其核心规则为不能直接实例化，必须通过非抽象子类实例化，且子类需重写所有抽象方法，否则子类也需定义为抽象类；要注意同一包下类名必须唯一，因为 Java 通过“包名 + 类名”唯一标识类，重名会导致编译失败。

2. 接口

接口是用 `interface` 修饰的“顶层设计”，仅包含抽象方法（默认 `public abstract`）和常量（默认 `public static final`），无构造方法。类实现接口需用 `implements` 关键字，且必须重写接口的所有抽象方法。一个类可实现多个接口，解决 Java “单继承”限制，但只能继承一个父类，接口的核心价值是“跨类统一规范”，适合定义不同类的共性方法，不过若接口未定义某方法，调用时需用 `instanceof` 判断类类型并向下转型才能调用接口外的方法，建议将共性方法加入接口以避免转型。

3. 重写的规范

重写是发生在父类与子类间的，子类对父类属性一致，返回值一致等方法的重写。同时注意子类与父类的构造方法执行有固定顺序，子类实例化时，优先执行父类构造方法，再执行子类构造方法，默认隐含 `super()` 调用父类无参构造。并且 `super()` 必须放在子类构造方法的第一行，且只能出现一次，这种执行顺序确保父类成员变量先初始化，子类可直接使用父类的初始化变量，如 `new SubClass()` 时，会先调用父类 `SuperClass` 的构造方法，再调用子类 `SubClass` 的构造方法。

4. 反射——Java 的动态性

它允许程序在运行时动态加载类、创建对象、调用方法，无需在编译期确定具体类。其核心步骤为通过类的全限定名获取类对象，即

```
Class<?> clazz = Class.forName(vehicleType);
```

判断类是否实现目标接口，用

```
if (ICommon.class.isAssignableFrom(clazz)) {
```

使用反射必须处理 Checked Exception，可在 main 方法后加 throws Exception 或用 try-catch 捕获，其核心优势是支持开闭原则，新增类时无需修改主程序，只需实现接口即可。

课堂知识回答：

1. 在接口中我们定义了 10 个抽象方法，但是最后我们只在其中实现了 8 个抽象方法，此时该如何表述？

在 Java 中，对于接口中未被实现的抽象方法（即没有默认实现的方法），如果一个类实现了该接口但未覆盖所有抽象方法，那么这个类必须被声明为抽象类——这是唯一的选择，没有其他替代方案。

原因是 Java 的语法规则规定：非抽象类（具体类）必须实现其实现的接口中所有的抽象方法。如果未完全实现，编译器会直接报错；而抽象类的特性就是可以包含未实现的抽象方法，因此只能通过将类声明为抽象类来规避编译错误。

2. 在异常处理时，在 catch 里中再出错会有什么结果

当 catch 块中再次抛出新异常时，当前 catch 块的后续代码会立即终止执行，新异常会沿调用栈向上传播以寻找最近的 try-catch 块捕获；若上层代码未对其处理，该异常会被 JVM 处理并导致程序崩溃（打印异常栈轨迹后终止），同时新异常会取代 try 块中抛出的原始异常，导致原始异常信息丢失（除非主动保留），可能给调试带来困扰。

实验感想：

很快，为期 8 周的 Java 学习之旅将要迎来尾声。我通过这 4 次实验深入浅出地学习了这门面向对象的语言，现在只剩下最后一次大作业了。在这次 Java 实验虽然过程中遇到了不少小问题，比如写抽象类时因为类名重复报错、用反射时忘记处理异常导致编译失败，但解决这些问题的过程反而让我印象更深刻——原来类名重复是因为 Java 靠“包名 + 类名”唯一识别类，反射必须处理异常是因为它属于运行时动态操作。通过亲手写抽象类、接口和反射代码，我不再是死记知识点，而是真正理解了抽象类怎么定义规范、接口怎么解决单继承限制、反射怎么实现“新增类不用改主程序”。这次实验也让我明白，编程不能只靠想，得多动手试，遇到报错别慌，查原因、改代码的过程就是进步的过程，以后写代码也会更注意类名规范和异常处理这些细节。